

Asaba Hustle Agent Handoff

This file is for another coding agent. It describes the current state of the repository as of March 11, 2026.

High-Level Summary

Asaba Hustle is a Laravel 12 local-services marketplace with:

- an API-first backend
- a Blade web client enhanced with Tailwind CSS and jQuery AJAX
- an admin panel for moderation and reporting
- Paystack and Flutterwave payment support
- Laravel Reverb for broadcast-ready real-time features

The project is no longer backend-only. It now has both:

- API routes for mobile/external clients
- web routes and web controllers for session-based browser usage

Current Stack

- Laravel 12
- PHP 8.2+
- Laravel Sanctum
- Spatie Laravel Permission

- Laravel Reverb
- Blade
- Tailwind CSS v4
- jQuery
- Vite
- SQLite for tests/local fallback
- MySQL or MariaDB for normal deployment

What Exists Now

API Layer

Implemented and currently routed:

- auth
- skills
- jobs
- messages
- notifications
- payments
- payment webhooks

The API response shape is standardized:

```
```json
```

```
{
```

```
"success": true,
"message": "...",
"data": {},
"meta": {}
}
` `` `
```

Validation, auth, authorization, and not-found errors are normalized for API consumers in  
`bootstrap/app.php`.

### ### Web Layer

The web layer was added on top of the existing backend and is intentionally JSON-first for  
AJAX interactions.

Current web pages/routes:

```
` `` `text
```

GET /

GET /login

POST /login

GET /register

POST /register

GET /verify-phone

POST /verify-phone

GET /forgot-password

POST /forgot-password

GET /reset-password

POST /reset-password

POST /logout

GET /app

GET /app/me

PUT /app/profile

POST /app/change-password

POST /app/upload-id

POST /app/availability

POST /app/send-verification-token

POST /app/verify-contact

GET /app/jobs

POST /app/jobs

GET /app/jobs/{job}

POST /app/jobs/{job}/apply

POST /app/jobs/{job}/hire

POST /app/jobs/{job}/accept

POST /app/jobs/{job}/start

POST /app/jobs/{job}/complete

POST /app/jobs/{job}/rate

GET /app/jobs/{job}/suggested-workers

GET /app/my-jobs

GET /app/conversations

GET /app/conversations/{conversation}/messages

POST /app/conversations/{conversation}/read

POST /app/messages

GET /app/notifications

POST /app/notifications/read

POST /app/notifications/read-all

POST /app/payments/paystack/initialize

POST /app/payments/paystack/verify

POST /app/payments/flutterwave/initialize

POST /app/payments/flutterwave/verify

...

Current admin routes:

```text

/admin/dashboard

/admin/users

/admin/jobs

/admin/payments

/admin/ratings

/admin/activity-logs

...

Browser Interaction Model

Important: the intended frontend pattern is not full-page form submission. The current direction is:

- Blade renders the page shell
- jQuery submits forms over AJAX
- controllers return JSON for those actions
- the frontend updates the page in place

This pattern is implemented in `resources/js/app.js`.

Key Files Added or Updated Recently

Web Controllers

- `app/Http/Controllers/Web/HomeController.php`
- `app/Http/Controllers/Web/AuthController.php`
- `app/Http/Controllers/Web/AppController.php`

Views

- `resources/views/layouts/app.blade.php`
- `resources/views/web/home.blade.php`
- `resources/views/web/app.blade.php`
- `resources/views/web/auth/login.blade.php`
- `resources/views/web/auth/register.blade.php`

- `resources/views/web/auth/verify-phone.blade.php`
- `resources/views/web/auth/forgot-password.blade.php`
- `resources/views/web/auth/reset-password.blade.php`

Frontend Assets

- `resources/js/app.js`
- `resources/css/app.css`

Routes

- `routes/web.php`

Existing API Behavior Updated

- `app/Http/Controllers/Api/AuthController.php`

Email verification now redirects browser users to the login page with query-string state instead of leaving them on a raw JSON response.

Tests Added

- `tests/Feature/Web/WebFrontendReadinessTest.php`

Current Frontend Behavior

Theme Tokens

Defined in `resources/css/app.css` :

- `--brand: #FF7A00`
- `--brand-strong: #E56700`
- `--ink: #333333`
- `--surface: #FFFFFF`
- `--surface-soft: #FFF6ED`

jQuery Usage

`jquery` is installed and imported in `resources/js/app.js`.

That file currently handles:

- CSRF-aware AJAX setup
- generic success/error feedback rendering
- AJAX auth form submission
- AJAX job list loading
- AJAX client-side job creation
- AJAX logout

Current App Screen

`resources/views/web/app.blade.php` currently provides:

- authenticated app shell
- profile and identity operations
- verification controls
- job feed and job lifecycle controls
- my-jobs snapshot
- conversation/message controls
- notifications controls
- payment initialize and verify controls
- AJAX configuration bootstrapped into `window.asabaAppConfig`

This is now a broad parity/control-surface implementation, not a polished end-user product UI.

Current Backend Behavior Worth Knowing

Auth

- Registration requires `verification\_method`
- If `verification\_method=email`, email is required
- Web login uses session auth
- API login uses Sanctum tokens
- Registration does not automatically log in the user

Jobs

- Policy check is still enforced on job creation
- Web app job lifecycle endpoints reuse the same service/policy model as the API
- Web app job list is driven from the same `ServiceJob` model and resource layer as the API

Admin

- Admin layout now uses `@vite` instead of the Tailwind CDN
- Admin pages are still simple Blade views and not yet AJAX-driven

Tests and Verification

Last confirmed locally:

- `php artisan test` passes
- total passing tests: 22

Included coverage now touches:

- API auth flows
- API hardening
- payment idempotency and webhooks
- admin filtering and moderation
- web login JSON endpoint
- web jobs JSON feed
- web profile update

- web job apply flow
- web notifications fetch

Important Operational Note

Do not assume a production asset build has been run.

The user explicitly said:

- do not build from the agent
- they will use `npm run dev` when they want to start the project

That means:

- `npm install` has been run
- `jquery` is installed
- you should not claim that `vite build` has been verified unless you run it later with explicit approval

Seeded Accounts

```text

Client

phone: 08000000010

password: password123

Worker

phone: 08000000020

password: password123

Pending phone verification user

phone: 08000000030

password: password123

token: 123456

Pending email verification user

phone: 08000000031

email: pending.email@asabahustle.test

password: password123

...

## ## Current Gaps

These are still open and should not be misrepresented as complete:

### ### Web UX

- no full production-grade public marketing site
- the `/app` screen is functional but still looks like an operator console more than a finished customer product
- some flows are exposed as raw control forms rather than polished contextual UI
- worker/client experiences still need role-specific refinement

### ### Payments

- no refund flow
- no escrow flow
- no polished browser-facing payment callback UX beyond initialization and verification endpoints

### ### Messaging

- browser messaging endpoints exist, but not yet as a refined chat product experience

### ### Notifications

- browser notification endpoints exist, but not yet as a refined notification center

### ### Ops

- cache invalidation is still mostly TTL-driven
- no structured observability layer
- queue and listener idempotency need deeper audit
- SMS delivery is still application-level and not integrated with a real provider

## ## Suggested Next Tasks For Another Agent

Pick one of these directions instead of mixing all of them at once:

## 1. Expand the web app UI

- replace control-surface forms with role-aware product screens
- add worker dashboard states
- add richer my-jobs and job-detail views
- make lifecycle actions contextual instead of ID-driven

## 2. Build browser chat

- render conversation list
- render conversation thread
- send messages via AJAX
- optionally layer Reverb real-time subscriptions on top

## 3. Build browser notifications

- add notification dropdown/page
- use current JSON endpoints

## 4. Improve web auth UX

- better inline validation rendering
- resend verification token/link flow
- profile editing pages

## 5. Harden production readiness

- cache invalidation
- better tests around job lifecycle edge cases
- queue/retry audit

- real SMS/email integrations

## ## Constraints and Direction

- Keep browser mutation endpoints JSON-first if they are intended for AJAX.
- Reuse existing request classes, resources, policies, and services when possible.
- Do not replace the API-first structure with controller-specific duplicated business logic.
- Preserve the current design token direction:
  - primary `#FF7A00`
  - secondary `#333333`
  - background `#FFFFFF`

## ## One-Line Status

The repository currently has a solid Laravel API, a working admin panel, and a near-parity AJAX-driven Blade web layer, but the web UI still needs a real product-design pass before it feels like a finished end-user application.